

Proyecto Unidad 2

Sistema Clínico

Licitación Cesfam Villa Alemana

Nombre: María Paz Espinoza y Juan Machuca Araya

Licitación y problema abordado

El objetivo de este proceso es la creación de una plataforma tecnológica que permita a los pacientes solicitar horas medicas vía telefónica y WEB de manera eficiente y segura, con integración directa con el sistema de registro clínico electrónico RAYEN, utilizado en los diferentes Centros de Salud Familiar (CESFAM) de la Corporación Municipal de Villa Alemana (CMVA) para la gestión de la información clínica de los pacientes.

En primera instancia creamos una versión preliminar que cumplía con lo solicitado en la licitación. Esta funcionaba en base a distintas maquinas que trabajaban en conjunto y de manera simultánea, permitiendo que los usuarios agendaran sus horas medicas de manera fácil, también permitía que el personal administrativo tuviera conciencia del flujo de pacientes en los CESFAM de la CMVA con la ayuda de estadísticas en base a las horas tomadas por los pacientes.

En esta nueva versión, se utilizó todo lo generado en el proyecto 1 y se migro completamente a Docker, manteniendo los requisitos, estos se pueden revisar (junto a toda la propuesta de unidad 1) en el documento adjunto para la propuesta de la unidad.

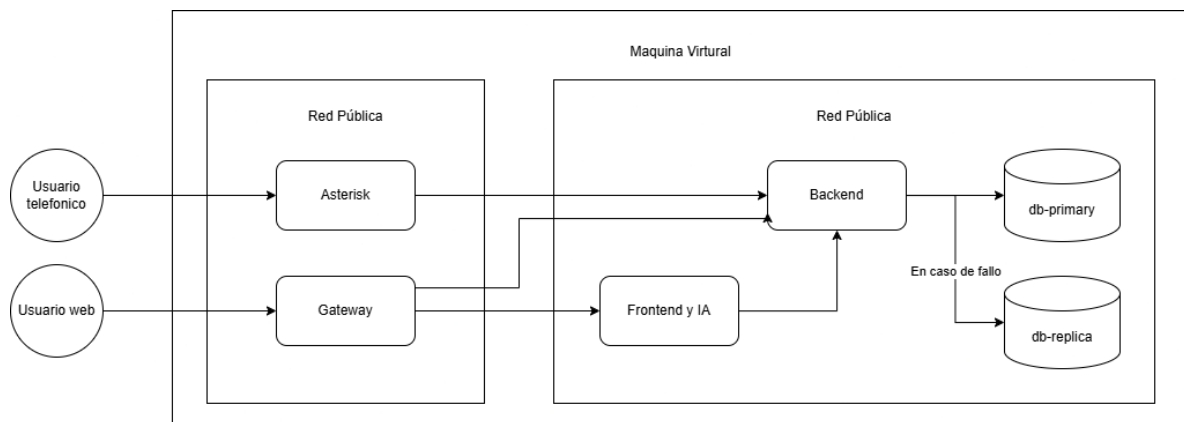
Arquitectura Contenerizada

En el diseño de esta nueva versión del sistema clínico, optamos por mantener la estructura de servicios desacoplados que veníamos trabajando, pero evolucionándolo hacia un modelo mucho más eficiente. En la unidad uno, cada servicio vivía en su propia máquina virtual independiente. Si bien esto nos permitía gestionar cada componente por separado y evitar colapsos masivos en caso de fallos, presentaba un problema importante de recursos ya que mantener 4 máquinas completas corriendo generaba un costo elevado y un desperdicio de recursos, sobre todo con aquellos servicios que tenían poco consumo.

Como solución a este problema de los costos sin perder los beneficios de tener todo por separado, decidimos abstraer la infraestructura utilizando contenedores de Docker. En lugar de usar múltiples máquinas completas con sus propios recursos, el uso de contenedores nos permitió empaquetar cada uno de los microservicios juntos con sus dependencias, haciéndolos convivir dentro de un mismo entorno.

Así logramos tener lo mejor de ambas partes, mantenemos el aislamiento entre las aplicaciones, pero optimizando el consumo de memoria y CPU al centralizarlo en una arquitectura mucho más ligera y moderna.

Para dar más contexto de la solución para la Unidad 2 explicaremos más a detalle la lógica de los contenedores y de la arquitectura utilizada en general.



Redes y comunicación

Al mover todos los microservicios a un único servidor surgió el problema de la seguridad al momento de comunicarlos con los usuarios y entre ellos mismos, ya que al mover información delicada (historiales médicos) debemos asegurar que esta no pase a los usuarios indebidos. Para resolverlo implementamos la lógica de redes segmentadas usando las redes virtuales internas de Docker, en lugar de abrir los puertos, creamos un entorno privado de comunicación asegurando un aislamiento total.

Hay 2 zonas definidas:

rayen_public_net, esta es la que se encarga de actuar como el perímetro expuesto al tráfico externo a interfaces controladas, permite mapear puertos hacia el host 80, 5060, 10000+, 5001. Los contenedores en esta red interactúan directamente con las solicitudes externas.

rayen_private_net, esta es la red hermética que restringe el acceso directo desde internet y el tráfico en general desde el exterior hacia sus contenedores. Esta es la lógica operativa del sistema, contiene:

Frontend: contenedor aislado en la red interna. No expone puertos al host, el tráfico entrante es otorgado por el gateway

Backend: contenedor crítico del sistema, procesa las solicitudes transaccionales. Opera en un entorno aislado y se parametriza dinámicamente mediante el consumo del archivo de las variables de entorno .env

Bases de datos: ambas bases de datos (primary y replica) viven exclusivamente en esta red privada. Son completamente invisibles desde el exterior, garantizando que la única forma de acceder a ellas sea a través del Backend.

Tecnologías

Ya mencionamos que el proyecto está dentro de contenedores, pero además utilizamos varias tecnologías para cumplir con lo que ya habíamos realizado en la unidad 1.

1. Node.js

Para el motor principal del sistema elegimos Node, al tratarse de un sistema de agendamiento, el servidor para mucho tiempo esperando respuestas (de las bases de datos, por ejemplo). En lugar de bloquearse, Node maneja las peticiones de entrada y salida de forma eficiente. Lo acompañamos con Express para levantar un API REST, estableciendo rutas para el frontend y para Asterisk.

2. MySQL

Para guardar la información manejada de los profesionales y pacientes optamos por utilizar MySQL. Elegimos una base de datos relacional porque la información médica está estructurada con conexiones obligatorias entre sus partes. Por ejemplo, un paciente tiene citas, una cita pertenece a un profesional, un profesional tiene un rol y especialidad, etc.

3. Asterisk

Uno de los requisitos de la licitación era la posibilidad de agendar citas médicas a través de un sistema telefónico. Utilizamos Asterisk ya que nos permitía manejar los protocolos SIP y RTP, además de su sistema IVR que es lo que nos entregaba la opción de capturar las teclas que el paciente ingresa y transformarlas en consultas directas al Backend.

4. Nginx

Elegimos Nginx para actuar como Gateway y Proxy inverso, en lugar de exponer el frontend y el servidor de Node directamente al internet. Nginx recibe las peticiones y decide hacia que contenedor las debe enrutar.

Monitoreo

Para el monitoreo del estado de cada contenedor y el uso de recursos de la máquina utilizamos healthchecks y tecnologías especializadas.

Si queremos saber el estado del contenedor, Docker consulta de manera constante un endpoint específicos que creamos en el backend (/api/health), si por algún motivo el servicio se bloquea o pierde conexión, el sistema lo marca automáticamente como unhealthy, permitiendo detectar fallos silenciosos en tiempo real antes de que afecten el agendamiento de pacientes.

Por otro lado, si queremos saber el uso de recursos de la máquina, podemos consultarlo en '/grafana', lo cual nos mostrará unos gráficos con el detalle gracias a:

Cadvisor, requiere un nivel de ejecución elevado, debido a que monta directorios clave del sistema de archivos del host. Analiza a nivel de kernel el consumo de la CPU, memoria, red y disco de todos los contenedores en tiempo de ejecución.

Prometheus: consume las métricas recolectadas por el cadvisor a través de la red privada de manera cíclica. Utilizando configuraciones estáticas declaradas en su archivo.

Grafana: actúa como la interfaz de visualización. A el tráfico se administra a través del .env. permitiendo que el contenedor sirva los paneles a través de una sub-ruta del gateway público, manteniendo la cohesión de acceso

Respaldo y recuperación

Se creó un Script de respaldo en Bash que se ejecuta de manera automática al levantar los contenedores, este script interactúa de manera directa con la base de datos primaria (primary). Utiliza la herramienta nativa de MySQL, mysqldump, para extraer copias exactas de nuestra base de datos principal, y las guarda en un directorio seguro y protegido del sistema operativo anfitrión. Este actúa todas las madrugadas de manera automática para evitar errores humanos.

Además, este script tiene una política de retención que elimina automáticamente los archivos con más de 7 días de antigüedad, optimizando el uso del almacenamiento del servidor.

Para recuperar los datos tenemos una estrategia de recuperación que opera en dos niveles.

Primero tenemos un clúster creado para el manejo de las peticiones a las bases de datos, si el nodo primario se cae, el nodo réplica contiene el estado exacto de la información hasta los últimos cambios gracias a la sincronización de los binlogs, permitiendo una transición rápida sin pérdida de datos.

En segundo lugar, en caso de una caída completa de las bases de datos o si los datos se corrompen, al tener los respaldos aislados, podemos reconstruir y restaurar la información completa en cuestión de minutos.

Inteligencia artificial y su aplicación

Se optó por utilizar ollama como inteligencia artificial, la cual, utiliza llama3 como lenguaje, se aplicó esta inteligencia artificial en un chatbot.

Su prompt está diseñado para limitar las respuestas fuera de contexto que un usuario podría hacerle al chatbot. Su función se limita a ayudar y guiar a los pacientes para agendar sus citas o para confirmar/anular sus citas ya agendadas con anterioridad.

Se instala mediante el contenedor de ollama_init, el cual descarga la versión de ollama 0.30.8.

La ejecución de ollama está completamente radicada en el contenedor chat-ia, este encapsula la lógica de interacción, para que el cliente final pueda utilizar la IA.

Procedimiento de Despliegue

Una de las consideraciones principales que tuvimos en cuenta fue eliminar el error humano al desplegar el sistema. En lugar de ejecutar comandos manuales para levantar cada microservicio por separado, consolidamos toda la topología en dos archivos `docker-compose.yml`. Estos definen de forma declarativa que imágenes usar, a que redes virtuales pertenece cada contenedor y cual es el orden exacto de arranque mediante reglas de dependencia. De esta forma, levantar la infraestructura completa se reduce a solo 2 comandos, uno para las bases de datos (`db-primary`, `db-replica` y `db-setup`) y otro para todo el resto de servicios y las redes virtuales.

Un aspecto crítico en el despliegue es el uso de contraseñas y credenciales, sabemos que nunca deben quedar escritas en el código fuente por lo que implementamos un único archivo `.env` en la raíz del proyecto. Durante el despliegue, Docker compose se encarga de leer este archivo e inyectar dinámicamente las variables correspondientes a cada contenedor.

A nivel de comandos solo usamos estos 2.

Primero, en la carpeta raíz tenemos un Docker compose, el cual se ejecuta de manera simple:

```
sudo docker compose up -d --build
```

Luego debemos ir a la carpeta de las bases de datos (`/infraestructura/databases`) y ejecutar casi el mismo comando, solo que debemos entregar la ubicación del archivo `.env` maestro:

```
Sudo docker compose --env-file ../../.env up -d
```